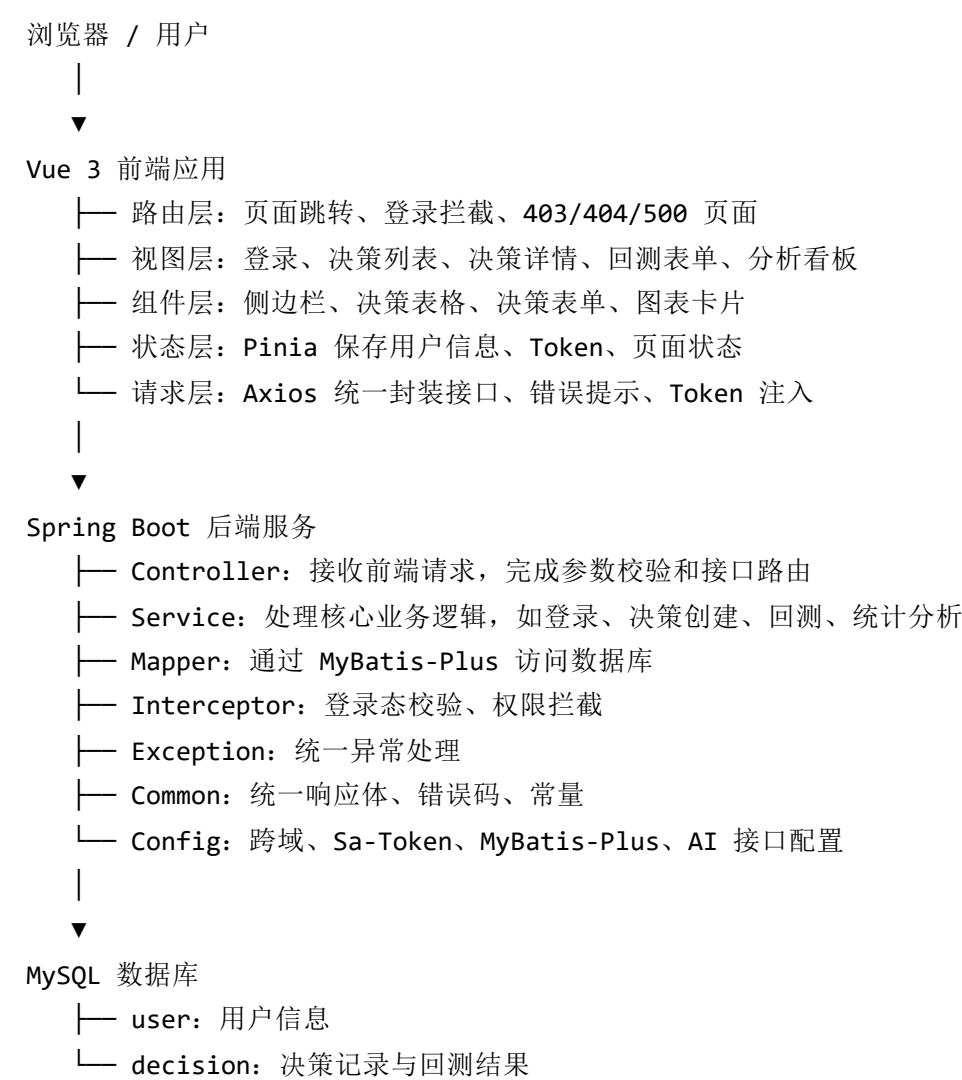


个人决策回测器架构文档

1. 整体架构

本项目根据《个人决策回测器 SpringBoot + Vue 版设计文档》设计为“前后端分离 + 后端单体分层 MVC + 前端模块化页面”的 Web 应用。系统由 Vue 3 前端应用、Spring Boot 后端服务和 MySQL 数据库三部分组成。



整体设计目标是保证系统职责清晰、模块边界明确、接口格式统一，便于期末验收时展示完整业务闭环：

- 注册 / 登录
- 记录一笔决策

→ 设置回测时间

→ 到期提交回测

→ 查看统计看板

→ 生成 AI 复盘建议

2. 技术选型

2.1 前端技术栈

技术	作用
Vue 3	前端主框架，使用 Composition API 组织页面逻辑
Vite	前端开发服务、热更新、生产构建
Element Plus	表单、按钮、弹窗、表格等基础 UI 组件
ECharts	满意度饼图、决策趋势折线图等可视化展示
Pinia	登录用户、Token、页面状态等全局状态管理
Vue Router	页面路由、登录拦截、错误页跳转
Axios	统一请求后端接口，封装错误处理和 Token 注入

前端采用模块化组织方式，将页面、组件、接口、状态、常量和工具函数拆分到不同目录，避免页面文件承担过多职责。

2.2 后端技术栈

技术	作用
Spring Boot 3.x	后端主框架，提供 REST API 服务
Maven	后端依赖管理、构建和打包
MyBatis-Plus	简化单表 CRUD、分页查询、自动填充等持久层操作
MySQL 8.0	存储用户、决策记录、回测结果等业务数据
Sa-Token	轻量级登录认证、Token 管理和接口鉴权

技术	作用
Jakarta Validation	Controller 入参校验
Lombok	简化 Entity、DTO、VO 等数据类代码
HTTP Client	调用第三方大模型接口生成 AI 建议

后端采用单模块 Spring Boot 应用，按业务模块组织代码。Controller 只负责接口入口，Service 负责业务闭环，Mapper 负责数据库访问，Common/Config/Exception 提供全局基础能力。

2.3 数据库技术

数据库选用 MySQL 8.0，主要包含两张核心表：

表名	作用
user	保存用户账号、密码、昵称、状态和时间字段
decision	保存决策标题、背景、候选方案、原因、心情、回测时间、满意度和反馈

数据库脚本统一放在 sql/ 目录中：

```
sql/
├─ schema.sql      # 建表脚本
└─ data.sql        # 预置测试账号和演示数据
```

3. 项目结构

项目目标结构如下：

```

decision-tracker/
├── .github/workflows/main.yml
├── frontend/                                     # 前端 Vue 3 + Vite 应用
│   ├── public/                                 # 静态资源
│   ├── src/
│   │   ├── api/                               # API 请求函数，按业务模块拆分
│   │   │   ├── auth.js                       # 登录、注册、用户信息接口
│   │   │   ├── decision.js                   # 决策记录增删改查接口
│   │   │   ├── analysis.js                   # 图表统计接口
│   │   │   └── advice.js                     # AI 建议接口
│   │   ├── assets/                           # 图片、图标等静态资源
│   │   ├── components/                       # 可复用组件
│   │   │   ├── AppSidebar/                   # 侧边栏菜单
│   │   │   ├── DecisionTable/               # 决策列表表格
│   │   │   ├── DecisionForm/                # 决策创建 / 编辑表单
│   │   │   └── ChartPanel/                  # 图表容器组件
│   │   ├── constants/                       # 常量配置
│   │   │   ├── apis.js                      # 后端接口路径常量
│   │   │   ├── urls.js                      # 前端路由地址常量
│   │   │   └── errorCodes.js                # 前端错误码映射
│   │   ├── hooks/                           # 组合式逻辑封装
│   │   │   ├── useAuth.js                   # 登录状态与权限判断
│   │   │   └── useRequest.js                # 请求 loading/error 封装
│   │   ├── router/                          # 路由配置
│   │   ├── store/                           # Pinia 状态管理
│   │   │   └── auth.js                      # 登录用户与 Token 状态
│   │   ├── utils/                           # 工具函数
│   │   │   ├── request.js                   # Axios 实例、拦截器、统一错误处理
│   │   │   └── token.js                     # Token 存取工具
│   │   ├── views/                           # 页面组件
│   │   │   ├── login/                       # 登录 / 注册页
│   │   │   ├── dashboard/                  # 数据分析看板
│   │   │   ├── decision-list/              # 决策列表页
│   │   │   ├── decision-detail/            # 决策详情与回测页
│   │   │   ├── forbidden/                  # 403 页面
│   │   │   ├── not-found/                  # 404 页面
│   │   │   └── server-error/                # 500 页面
│   │   ├── App.vue                          # 根组件
│   │   └── main.js                          # 前端入口
│   ├── .env                                 # 前端环境变量
│   ├── .env.example                         # 前端环境变量示例
│   ├── package.json                         # 前端依赖配置
│   └── vite.config.js                       # Vite 配置

```

```
|
|
|─ backend/                                # 后端 Spring Boot 3.x 应用
|   |
|   |─ src/
|   |   |
|   |   |─ main/
|   |   |   |
|   |   |   |─ java/com/decision/
|   |   |   |   |
|   |   |   |   |─ DecisionApplication.java
|   |   |   |   |─ config/                # 全局配置
|   |   |   |   |   |
|   |   |   |   |   |─ CorsConfig.java
|   |   |   |   |   |─ SaTokenConfig.java
|   |   |   |   |   |─ MybatisPlusConfig.java
|   |   |   |   |   └─ AiClientConfig.java
|   |   |   |   |─ common/                # 通用基础能力
|   |   |   |   |   |
|   |   |   |   |   |─ Result.java
|   |   |   |   |   |─ PageResult.java
|   |   |   |   |   |─ ErrorCode.java
|   |   |   |   |   |─ BusinessException.java
|   |   |   |   |   └─ GlobalExceptionHandler.java
|   |   |   |   |─ constants/            # 系统常量
|   |   |   |   |   |
|   |   |   |   |   |─ AuthConstants.java
|   |   |   |   |   |─ DecisionConstants.java
|   |   |   |   |   └─ ErrorMessageConstants.java
|   |   |   |   |─ auth/                 # 认证与登录模块
|   |   |   |   |   |
|   |   |   |   |   |─ controller/
|   |   |   |   |   |   └─ AuthController.java
|   |   |   |   |   |─ service/
|   |   |   |   |   |   |
|   |   |   |   |   |   |─ AuthService.java
|   |   |   |   |   |   └─ impl/AuthServiceImpl.java
|   |   |   |   |   |─ dto/
|   |   |   |   |   |   |
|   |   |   |   |   |   |─ LoginRequest.java
|   |   |   |   |   |   └─ RegisterRequest.java
|   |   |   |   |   └─ vo/
|   |   |   |   |       |
|   |   |   |   |       |─ LoginResponse.java
|   |   |   |   |       └─ UserInfoVO.java
|   |   |   |   |─ user/                 # 用户基础信息模块
|   |   |   |   |   |
|   |   |   |   |   |─ entity/User.java
|   |   |   |   |   |─ mapper/UserMapper.java
|   |   |   |   |   |─ service/
|   |   |   |   |   |   |
|   |   |   |   |   |   |─ UserService.java
|   |   |   |   |   |   └─ impl/UserServiceImpl.java
|   |   |   |   |   └─ vo/UserVO.java
|   |   |   |   |─ decision/            # 决策核心模块
|   |   |   |   |   |
|   |   |   |   |   |─ entity/Decision.java
|   |   |   |   |   └─ controller/DecisionController.java
```

```

├── service/
│   ├── DecisionService.java
│   └── impl/DecisionServiceImpl.java
├── mapper/DecisionMapper.java
├── dto/
│   ├── DecisionCreateRequest.java
│   ├── DecisionPageRequest.java
│   └── DecisionReviewRequest.java
├── vo/
│   ├── DecisionDetailVO.java
│   └── DecisionPageVO.java
├── analysis/           # 数据分析与看板模块
│   ├── controller/AnalysisController.java
│   ├── service/
│   │   ├── AnalysisService.java
│   │   └── impl/AnalysisServiceImpl.java
│   └── vo/
│       ├── SatisfactionPieVO.java
│       └── TrendLineVO.java
├── advice/             # AI 建议生成模块
│   ├── controller/AdviceController.java
│   ├── service/
│   │   ├── AdviceService.java
│   │   └── impl/AdviceServiceImpl.java
│   ├── client/AiAdviceClient.java
│   ├── dto/AdviceGenerateRequest.java
│   └── vo/AdviceVO.java
├── health/             # 健康检查接口
│   └── HealthController.java
├── resources/
│   ├── application.yml
│   └── application-example.yml
└── pom.xml

├── sql/                # 数据库脚本
│   ├── schema.sql
│   └── data.sql
├── scripts/            # 初始化与辅助脚本
├── docs/               # API、错误码、部署等文档
├── docker-compose.yml  # 容器化部署编排
└── README.md

```

4. 前端模块划分

4.1 Views 页面模块

页面模块按业务场景拆分：

页面	职责
login/	用户登录、注册、验证码输入
dashboard/	数据分析看板，展示统计图表和核心指标
decision-list/	决策列表、分页查询、标题模糊查询、标签和紧急度筛选
decision-detail/	决策详情、回测表单、AI 建议展示
forbidden/	403 无权限页面
not-found/	404 页面
server-error/	500 服务异常页面

4.2 Components 组件模块

组件模块沉淀可复用 UI：

组件	职责
AppSidebar	应用侧边栏、菜单导航
DecisionTable	决策列表表格、操作入口
DecisionForm	决策创建和编辑表单
ChartPanel	图表容器，承载 ECharts 图表

4.3 API 请求模块

前端接口函数按业务拆分：

文件	职责
auth.js	注册、登录、获取当前用户、退出登录

文件	职责
decision.js	创建决策、分页查询、详情、回测、删除
analysis.js	满意度饼图、决策趋势折线图
advice.js	请求 AI 决策建议

所有接口路径统一维护在 constants/apis.js 中，页面不直接写死后端 URL。

4.4 Store 状态模块

Pinia 主要维护：

- 当前登录用户信息。
- 当前 Token。
- 登录状态。
- 页面中需要跨组件共享的轻量状态。

Token 的持久化由 utils/token.js 统一处理，请求时由 utils/request.js 自动注入 Header。

4.5 Router 路由模块

Vue Router 负责：

- 页面跳转。
- 登录态校验。
- 未登录访问业务页面时跳转到登录页。
- 403、404、500 页面兜底。

5. 后端模块划分

5.1 Config 配置模块

config/ 负责系统基础配置：

- CORS 跨域配置。
- Sa-Token 登录认证配置。
- MyBatis-Plus 配置。
- AI 接口客户端配置。
- 其他全局 Bean 配置。

5.2 Common 与 Exception 模块

common/ 与统一异常处理模块负责后端基础规范：

- Result<T>：统一响应结构，包含 code 、 message 、 data 。
- ErrorCode：统一错误码。
- BusinessException：业务异常。
- GlobalExceptionHandler：使用 @RestControllerAdvice 统一处理异常。

统一响应格式示例：

```
{
  "code": 0,
  "message": "success",
  "data": {}
}
```

5.3 Auth 认证模块

认证模块负责用户身份相关能力：

- 用户注册。
- 用户登录。
- 获取当前登录用户信息。
- 用户退出登录。
- Token 生成、校验和注销。
- 登录态拦截和权限校验。

设计接口：

方法	路径	说明
POST	/api/auth/register	用户注册
POST	/api/auth/login	用户登录，调用 Sa-Token 登录并返回 Token
GET	/api/auth/info	获取当前登录用户信息
POST	/api/auth/logout	退出登录

5.4 User 用户模块

用户模块负责用户基础数据：

- 用户实体 User。
- 用户表 Mapper。
- 根据用户名查询用户。
- 保存注册用户。
- 管理账号状态。

用户表核心字段包括：

- id
- username
- password
- nickname
- create_time
- update_time

5.5 Decision 决策模块

决策模块是核心业务模块，负责：

- 创建新决策。
- 分页条件查询。
- 查看决策详情。
- 提交回测结果。
- 删除决策。
- 校验决策归属，确保用户只能访问自己的数据。

设计接口：

方法	路径	说明
POST	/api/decision	创建新决策
GET	/api/decision/page	分页条件查询，支持标题、标签、紧急度筛选
GET	/api/decision/{id}	获取决策详情
PUT	/api/decision/{id}/review	提交回测结果
DELETE	/api/decision/{id}	删除决策

决策表核心字段包括：

- id
- user_id
- title
- context
- options
- reason
- mood
- review_time
- satisfaction
- feedback
- create_time
- update_time

5.6 Analysis 数据分析模块

分析模块为前端 ECharts 提供数据支撑：

方法	路径	说明
GET	/api/analysis/satisfaction-pie	统计当前用户满意度分布
GET	/api/analysis/trend-line	按月统计决策数量趋势

该模块只负责聚合统计数据，不直接处理页面展示。前端拿到统计结果后交给 ECharts 渲染。

5.7 Advice AI 建议模块

AI 建议模块负责生成复盘建议：

方法	路径	说明
POST	/api/advice/generate/{decisionId}	根据某笔决策生成 AI 建议

核心流程：

- 查询当前决策
- 校验决策属于当前用户
 - 查询用户历史满意度分布
 - 拼接大模型 Prompt
 - 调用第三方 AI 接口
 - 返回结构化建议文本

5.8 Health 健康检查模块

健康检查模块用于部署和验收：

方法	路径	说明
GET	/api/health	返回服务状态、当前时间、数据库连接状态

6. 数据库设计

6.1 用户表 user

字段	说明
id	用户 ID
username	用户名，唯一
password	密码密文
nickname	用户昵称
create_time	创建时间
update_time	更新时间

6.2 决策表 decision

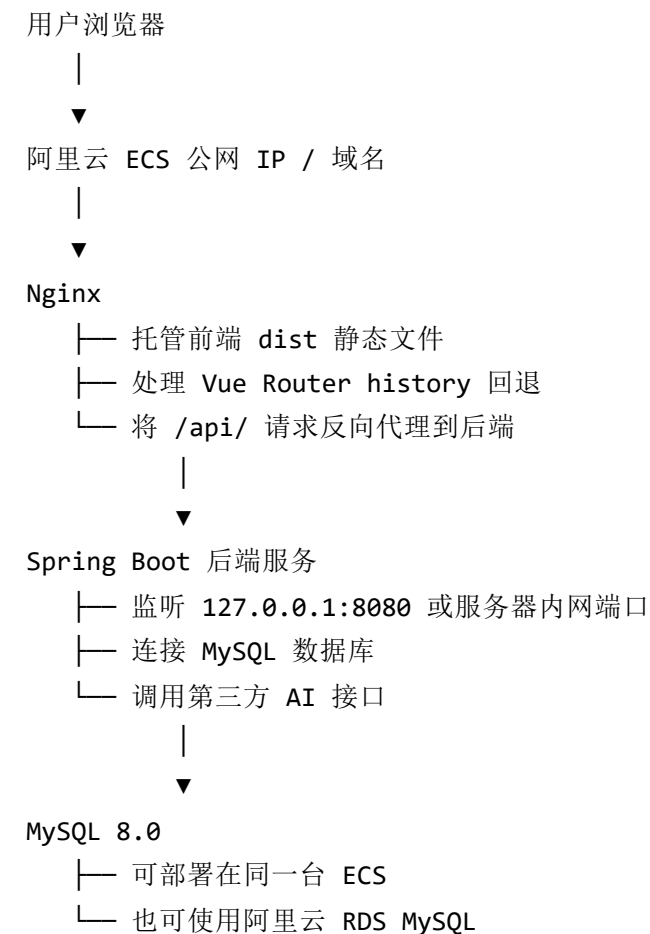
字段	说明
id	决策 ID
user_id	关联用户 ID

字段	说明
title	决策标题
context	决策背景 / 场景
options	候选方案
reason	做出该决策的核心原因
mood	做决策时的情绪
review_time	计划回测复盘时间
satisfaction	回测满意度
feedback	回测文字复盘
create_time	创建时间
update_time	更新时间

7. 阿里云服务器部署方式

7.1 部署架构

项目部署到阿里云 ECS 云服务器，采用“公网 Nginx 统一入口 + Spring Boot 后端 Jar + MySQL 数据库”的方式。



推荐用于期末项目的简化部署方式是：前端、后端、MySQL 均部署在同一台 ECS；如果需要更稳定的生产环境，可将 MySQL 替换为阿里云 RDS。

7.2 阿里云资源准备

需要准备以下资源：

资源	说明
ECS 云服务器	建议使用 Ubuntu 22.04 LTS 或 CentOS Stream
安全组	开放 80 、 443 、 22 端口；后端端口只允许本机访问
公网 IP / 域名	用于访问前端页面
MySQL 8.0 或 RDS	存储业务数据
JDK 17	运行 Spring Boot 后端
Node.js	用于服务器构建前端，或本地构建后上传

资源	说明
Maven	用于服务器构建后端，或本地构建后上传
Nginx	托管前端静态资源并反向代理后端

安全组建议：

- 22：SSH 登录，仅允许本人 IP 更安全。
- 80：HTTP 访问。
- 443：HTTPS 访问，配置证书后使用。
- 8080：不建议对公网开放，后端由 Nginx 反向代理访问。
- 3306：如果 MySQL 部署在同一台 ECS，不建议对公网开放。

7.3 服务器目录规划

服务器上建议使用固定目录管理项目：

```
/opt/decision-tracker/
├─ frontend/
│   └─ dist/                # 前端构建产物
├─ backend/
│   ├── app.jar             # 后端 Jar 包
│   ├── logs/               # 后端日志
│   └─ application-prod.yml # 生产配置
├─ sql/
│   ├── schema.sql
│   └─ data.sql
└─ backup/                  # 数据库备份文件
```

7.4 数据库部署

如果 MySQL 部署在 ECS 上：

```
sudo apt update
sudo apt install mysql-server -y
sudo systemctl enable mysql
sudo systemctl start mysql
```

初始化数据库：

```
mysql -u root -p < /opt/decision-tracker/sql/schema.sql
mysql -u root -p < /opt/decision-tracker/sql/data.sql
```

建议创建独立业务账号，不直接使用 root：

```
CREATE USER 'decision_user'@'localhost' IDENTIFIED BY 'your_password';
GRANT ALL PRIVILEGES ON decision_tracker.* TO 'decision_user'@'localhost';
FLUSH PRIVILEGES;
```

如果使用阿里云 RDS MySQL，则在 RDS 控制台创建数据库和账号，并在后端生产配置中填写 RDS 内网地址、端口、用户名和密码。

7.5 后端部署

后端在本地或服务器执行构建：

```
cd backend
mvn clean package -DskipTests
```

上传 Jar 到服务器：

```
/opt/decision-tracker/backend/app.jar
```

生产配置文件建议单独放置：

```
/opt/decision-tracker/backend/application-prod.yml
```

配置内容示例：


```
server:
  port: 8080

spring:
  datasource:
    url: jdbc:mysql://127.0.0.1:3306/decision_tracker?useUnicode=true&characterEncoding=utf8&se
    username: decision_user
    password: your_password

sa-token:
  token-name: Authorization
  timeout: 86400
  active-timeout: -1
  is-concurrent: true
  token-style: uuid
```

使用 systemd 管理后端服务：

```
[Unit]
Description=Decision Tracker Backend
After=network.target

[Service]
WorkingDirectory=/opt/decision-tracker/backend
ExecStart=/usr/bin/java -jar /opt/decision-tracker/backend/app.jar --spring.config.location=/opt
Restart=always
RestartSec=5
StandardOutput=append:/opt/decision-tracker/backend/logs/app.log
StandardError=append:/opt/decision-tracker/backend/logs/error.log

[Install]
WantedBy=multi-user.target
```

保存为：

```
/etc/systemd/system/decision-backend.service
```

启动服务：

```
sudo systemctl daemon-reload
sudo systemctl enable decision-backend
sudo systemctl start decision-backend
sudo systemctl status decision-backend
```

7.6 前端部署

前端在本地或服务器执行构建：

```
cd frontend
npm install
npm run build
```

将构建后的 `dist` 上传到：

```
/opt/decision-tracker/frontend/dist
```

前端生产环境请求统一走同域名 `/api`，由 Nginx 转发到后端，不需要把后端真实端口暴露给浏览器。

7.7 Nginx 配置

安装 Nginx：

```
sudo apt install nginx -y
sudo systemctl enable nginx
sudo systemctl start nginx
```

站点配置示例：

```
server {  
    listen 80;  
    server_name your-domain.example;  
  
    root /opt/decision-tracker/frontend/dist;  
    index index.html;  
  
    location / {  
        try_files $uri $uri/ /index.html;  
    }  
  
    location /api/ {  
        proxy_pass http://127.0.0.1:8080;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
}
```

启用配置后检查并重载：

```
sudo nginx -t  
sudo systemctl reload nginx
```

7.8 HTTPS 配置

如果绑定域名，建议在阿里云申请免费 SSL 证书或使用 Certbot 配置 HTTPS。

HTTPS 部署后：

- Nginx 监听 443。
- HTTP 80 自动重定向到 HTTPS。
- 前端仍通过 /api 访问后端。
- 后端服务继续只监听本机端口。

7.9 部署验证

部署完成后按以下顺序验证：

1. 访问 `http://公网IP` 或绑定域名，确认前端页面正常打开。

2. 请求 `/api/health`，确认后端服务正常。
3. 使用测试账号登录，确认 Token 能正常返回和保存。
4. 创建一条决策，确认数据库写入正常。
5. 提交回测，确认满意度统计能刷新。
6. 如配置 AI Key，验证 AI 建议接口能返回结果。

常用排查命令：

```
sudo systemctl status decision-backend
sudo tail -n 100 /opt/decision-tracker/backend/logs/app.log
sudo tail -n 100 /opt/decision-tracker/backend/logs/error.log
sudo nginx -t
sudo journalctl -u nginx -n 100
```

7.10 配置管理

部署配置应按环境拆分：

- `application.yml`：后端默认配置。
- `application-example.yml`：配置模板，不包含真实密码和 AI Key。
- `.env`：前端环境变量。
- `.env.example`：前端环境变量模板。

生产环境敏感信息包括：

- MySQL 用户名和密码。
- Sa-Token 密钥。
- AI 平台 API Key。

这些信息不应直接提交到代码仓库，应通过环境变量、服务器配置文件或部署平台密钥管理能力注入。